

Vistas indexadas en SQL Server

En la clase anterior vimos que una vista es una consulta guardada con nombre que se ejecuta cada vez que alguien la consulta. Eso es genial por su simplicidad, pero tiene una implicación que quizás no pensaste: si la vista `VW_VentasPorPaís` recibe quinientas consultas por hora, eso significa que SQL Server está ejecutando quinientas veces el mismo GROUP BY con cuatro JOINS. Cada vez desde cero. Cada vez recorriendo las mismas tablas, haciendo los mismos cálculos, devolviendo prácticamente el mismo resultado.

Cuando tienes pocos usuarios eso no se nota. Pero cuando la frecuencia de consultas sube, empiezan a pasar dos cosas al mismo tiempo: por un lado, el servidor está ocupado respondiendo todas esas consultas sobre las tablas; por otro lado, hay usuarios que están ingresando datos nuevos en esas mismas tablas. Y ahí se genera una tensión — lo que en arquitectura de datos se llama un **trade-off** — donde las lecturas y las escrituras compiten por los mismos recursos. En ese escenario, las vistas normales se quedan cortas.

Para eso existen las **vistas indexadas**, que en SQL Server son el equivalente a lo que otras bases de datos llaman vistas materializadas. La idea es que en vez de ejecutar la consulta cada vez, el resultado queda **precalculado y almacenado**, y SQL Server se encarga de mantenerlo actualizado automáticamente cuando los datos cambian.

Qué cambia respecto a una vista normal

Con una vista normal, lo que se guarda es la definición de la consulta — el SELECT con sus JOINS y filtros — y cada vez que alguien la consulta, SQL Server ejecuta toda esa lógica en el momento. No hay datos almacenados.

Con una vista indexada, el resultado de esa consulta sí se almacena físicamente. Los datos viajan desde las tablas originales hacia un espacio donde quedan disponibles para las consultas sin necesidad de recalcularlo todo. Eso significa que cuando alguien consulta la vista, SQL Server no tiene que ir a las tablas base, hacer los JOINS ni las agregaciones — simplemente lee el resultado que ya tiene guardado. La respuesta es mucho más rápida.

Pero ese beneficio tiene un costo. Cada vez que alguien inserta, actualiza o elimina datos en las tablas originales, SQL Server tiene que actualizar también la vista

indexada para que siga reflejando la realidad. Eso agrega trabajo a las operaciones de escritura. Es un intercambio: ganas velocidad en las lecturas a cambio de un poco más de costo en las escrituras.

Cómo se crea: los dos pasos obligatorios

Crear una vista indexada no es tan simple como crear una vista normal. SQL Server tiene requisitos estrictos, y tiene sentido que los tenga: si va a comprometerse a mantener datos precalculados sincronizados con las tablas originales, necesita ciertas garantías.

El primer paso es crear la vista con la cláusula `WITH SCHEMABINDING`. Esta es la parte que más confusión genera, así que vale la pena entenderla bien. Lo que SCHEMABINDING le dice a SQL Server es: "esta vista depende de estas tablas específicas, así que no permitas que nadie las modifique de una forma que rompa la vista". En la práctica, eso significa que si tu vista usa la columna `Total` de la tabla `Pedidos`, nadie va a poder eliminar ni alterar esa columna mientras la vista exista. SQL Server bloquea la operación y te avisa que hay un objeto que depende de ella.

En la clase lo vimos en vivo: el instructor intentó ejecutar `ALTER TABLE Pedidos DROP COLUMN Total` y SQL Server lo rechazó inmediatamente. Esa es la protección del SCHEMABINDING en acción. Es una restricción fuerte, pero es precisamente lo que garantiza que la vista indexada nunca se quede apuntando a algo que ya no existe.

Un detalle técnico que SCHEMABINDING exige: todas las referencias a tablas dentro de la vista deben incluir el schema completo. Es decir, no puedes escribir `FROM Pedidos`, tienes que escribir `FROM dbo.Pedidos`. Si no lo haces, la vista no se crea.

El segundo paso es crear un **índice clustered único** sobre la vista. Este es el paso que realmente materializa los datos. Hasta que no creas este índice, la vista con SCHEMABINDING es solo una vista normal con restricciones adicionales. El índice es lo que le dice a SQL Server: "ahora sí, almacena el resultado y mantenlo actualizado".

Cuándo vale la pena y cuándo no

Esta es la pregunta más importante de la clase, y la respuesta no es técnica sino de contexto. Una vista indexada no es algo que se aplica por defecto — es una decisión de arquitectura que depende de cómo se usan los datos en tu empresa.

Vale la pena cuando la consulta se ejecuta con mucha frecuencia — decenas o cientos de veces por hora — y los datos de las tablas base no cambian constantemente. También cuando la consulta tiene agregaciones pesadas (sumas, promedios, conteos) sobre tablas grandes, y cuando el tiempo de respuesta es crítico para el negocio, como en un dashboard que necesita cargar rápido.

No vale la pena cuando las tablas base reciben muchas escrituras todo el tiempo — muchos INSERTs, UPDATEs o DELETEs — porque el costo de mantener la vista actualizada puede terminar siendo mayor que el beneficio de tenerla precalculada. Tampoco tiene sentido si la consulta se ejecuta pocas veces al día, porque el ahorro es mínimo.

Para TiendaLatam, la vista de resumen de ventas por país es un buen candidato si los reportes se consultan frecuentemente y los pedidos completados rara vez se modifican (lo cual tiene sentido: una vez que un pedido está completado, generalmente no cambia). En cambio, una vista sobre inventario donde el stock se actualiza con cada venta probablemente no sea buena candidata, porque la frecuencia de escritura es demasiado alta.

La regla que plantea el instructor es clara: **mide primero, indexa después, nunca al revés**. No materialices vistas porque "por si acaso". Primero identifica que hay un problema real de rendimiento, y después evalúa si la vista indexada es la solución correcta.

Desafío de cierre

De las siguientes vistas de TiendaLatam, identifica cuáles serían buenas candidatas para ser indexadas y cuáles no: **VW_VentasPorPaís** (consultada diariamente para reportes), **VW_ClientesSinPedido** (consultada semanalmente para auditoría), **VW_ProductosTopVentas** (usada en un dashboard en tiempo real) y **VW_InventarioCritico** (stock menor a 10 unidades, que se actualiza con cada venta). Justifica cada decisión pensando en la frecuencia de consulta y la frecuencia de escritura sobre las tablas base. Comparte tu análisis en los comentarios de la clase.